

Subject:

PHP with MySQL

BCA 5th Semester

Course Code: BCAC501

Comprehensive notes provided by

Great Heights

Build | Believe | Become



Basic PHP Syntax

- PHP code is written inside `<?php ... ?>` tags.
- Statements end with a semicolon (`;`).
- PHP is case-insensitive for keywords (echo, if, etc.) but case-sensitive for variable names.

Example:

Basic PHP Script

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo 'Hello World!';
?>

</body>
</html>
```

PHP Statements

A statement is an instruction given to PHP.

```
<?php
$name = "John"; // Assignment statement
echo $name;     // Output statement
?>
```

echo

echo is used to output data.

```
<?php
echo "Hello Students";
?>
```

Output:

Hello Students

Comments in PHP

PHP supports two types of comments:

1. Single-line comments

```
<?php
// This is a single-line comment using double slashes
echo "Hello World!"; // Output greeting

# This is also a single-line comment using hash
echo "PHP Comments Example";
?>
```

2. Multiline comments

```
<?php
/*
This is a multi-line comment.
It can span across multiple lines.
Useful for detailed explanations.
*/
echo "Learning PHP comments!";
?>
```

PHP is Case-Sensitive

PHP variable names are case-sensitive.

Example:

```
$name = "Rahul";
```

and:

```
$Name = "Priya";
```

are two different variables.

```
echo $name;
```

```
echo $Name;
```

can produce different results.

PHP Variables

A variable in PHP is a container used to store data such as numbers, strings, arrays, or objects. The value stored in a variable can be changed or updated during the execution of the script.

- All variable names start with a dollar sign (\$).
- Variables can store different data types, like integers, strings, arrays, etc.
- PHP is loosely typed, so you don't need to declare a data type explicitly.
- Variable values can change during the script's execution.

Syntax: `$variable_name = value;`

Example:

```
<?php
$name = "XYZ"; // String
$age = 30;      // Integer
$salary = 45000.50; // Float
$isEmployed = true; // Boolean
?>
```

Example of Valid and Invalid Variable Names:

```
<?php

$firstName = "Alice"; // Valid
$_age = 25;           // Valid

$2ndPlace = "Bob";    // Invalid: Cannot start with a number
$class = "Physics";   // Valid, but avoid reserved words

?>
```

Variable Scope in PHP

PHP has **three main scopes**:

1. **Local** – inside a function.
2. **Global** – declared outside functions.
3. **Static** – retains value between function calls.

Example:

```
<?php
$globalVar = "I am global";

function testScope() {
    global $globalVar; // Access global variable
    echo $globalVar;
}

testScope();
?>
```

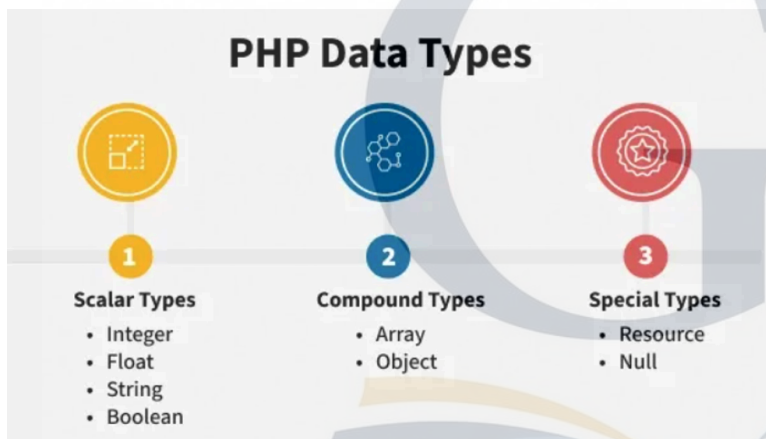
Variable Variables

```
<?php
$varName = "fruit";
$$varName = "Apple"; // Creates $fruit

echo $fruit; // Output: Apple
?>
```

Data Types

In PHP, data types define the kind of value a variable can hold. PHP is a loosely typed language, meaning you don't need to declare the data type of a variable. It is automatically assigned based on the value.



1. Scaler Types

1. Integer

Whole numbers (positive, negative, or zero).

Example:

```
$age = 25; // integer
```

2. Float / Double

Numbers with decimal points or in exponential form.

Example:

```
$price = 99.99; // float
```

3. String

Sequence of characters enclosed in quotes.

Example:

```
$name = "John Doe"; // string
```

4. Boolean

Represents true or false.

Example:

```
$isActive = true; // boolean
```

2. Compound Types (hold multiple values)

1. Array

Collection of values (indexed or associative).

Example:

```
$colors = ["red", "green", "blue"]; // indexed array  
$person = ["name" => "Alice", "age" => 30]; // associative array
```

2. Object

Instance of a class containing properties and methods.

Example:

```
class Car {  
    public $brand;  
    function __construct($brand) {  
        $this->brand = $brand;  
    }  
}  
$car = new Car("Toyota");
```

3. Callable

A variable that can be called as a function.

Example:

```
function greet() { echo "Hello!"; }  
$func = 'greet';  
$func(); // calls greet()
```

4. Iterable

Any value that can be looped over with foreach (arrays, objects implementing Traversable).

3. Special Types

1. NULL

Represents a variable with no value.

Example:

```
$x = null;
```

2. Resource

Holds a reference to an external resource (e.g., database connection, file handle).

Example:

```
$file = fopen("test.txt", "r");
```


PHP Operators

In PHP, operators are special symbols used to perform operations on variables and values. Operators help you perform a variety of tasks, such as mathematical calculations, string manipulations, logical comparisons, and more.

1. Arithmetic Operators

Arithmetic operators are used to perform basic arithmetic operations like addition, subtraction, multiplication, division, and modulus.

Operator	Name	Syntax	Operation
+	Addition	$\$x + \y	Sum the operands
-	Subtraction	$\$x - \y	Differences in the Operands
*	Multiplication	$\$x * \y	Product of the operands
/	Division	$\$x / \y	The quotient of the operands
**	Exponentiation	$\$x ** \y	$\$x$ raised to the power $\$y$
%	Modulus	$\$x \% \y	The remainder of the operands

Example:

```
<?php
// Define two numbers
$x = 10;
$y = 3;

// Addition
echo "Addition: " . ($x + $y) . "\n";

// Subtraction
echo "Subtraction: " . ($x - $y) . "\n";

// Multiplication
echo "Multiplication: " . ($x * $y) . "\n";

// Division
echo "Division: " . ($x / $y) . "\n";

// Exponentiation
echo "Exponentiation: " . ($x ** $y) . "\n";

// Modulus
echo "Modulus: " . ($x % $y) . "\n";
?>
```

Output

```

Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333
Exponentiation: 1000
Modulus: 1
  
```

2. Logical Operators

Logical operators are used to operate with conditional statements. These operators evaluate conditions and return a boolean result (true or false).

Operator	Name	Syntax	Operation
and	Logical AND	\$x and \$y	True if both the operands are true else false
or	Logical OR	\$x or \$y	True if either of the operands is true otherwise, it is false
xor	Logical XOR	\$x xor \$y	True if either of the operands is true and false if both are true
&&	Logical AND	\$x && \$y	True if both the operands are true else false
	Logical OR	\$x \$y	True if either of the operands is true otherwise, it is false
!	Logical NOT	!\$x	True if \$x is false

Example:

```

<?php
$x = 50;
$y = 30;
if ($x == 50 and $y == 30)
    echo "and Success \n";

if ($x == 50 or $y == 20)
    echo "or Success \n";

if ($x == 50 xor $y == 20)
    echo "xor Success \n";

if ($x == 50 && $y == 30)
    echo "&& Success \n";

if ($x == 50 || $y == 20)
    echo "|| Success \n";

if (!$z)
    echo "! Success \n";
?>
  
```


Output

```

and Success
or Success
xor Success
&& Success
|| Success
! Success
  
```

3. Comparison Operators

Comparison operators are used to compare two values and return a Boolean result (true or false).

Example:

Operator	Name	Syntax	Operation
==	Equal To	\$x == \$y	Returns True if both the operands are equal
!=	Not Equal To	\$x != \$y	Returns True if both the operands are not equal
<>	Not Equal To	\$x <> \$y	Returns True if both the operands are unequal
===	Identical	\$x === \$y	Returns True if both the operands are equal and are of the same type
!==	Not Identical	\$x !== \$y	Returns True if both the operands are unequal and are of different types
<	Less Than	\$x < \$y	Returns True if \$x is less than \$y
>	Greater Than	\$x > \$y	Returns True if \$x is greater than \$y
<=	Less Than or Equal To	\$x <= \$y	Returns True if \$x is less than or equal to \$y
>=	Greater Than or Equal To	\$x >= \$y	Returns True if \$x is greater than or equal to \$y

```
<?php
$a = 80;
$b = 50;
$c = "80";

// Here var_dump function has been used to
// display structured information. We will learn
// about this function in complete details in further
// articles.
var_dump($a == $c) . "\n";
var_dump($a != $b) . "\n";
var_dump($a <> $b) . "\n";
var_dump($a === $c) . "\n";
var_dump($a !== $c) . "\n";
var_dump($a < $b) . "\n";
var_dump($a > $b) . "\n";
var_dump($a <= $b) . "\n";
var_dump($a >= $b);
?>
```

Output

```
bool(true)

Warning: A non-numeric value encountered in /home/guest/sandbox/Solution.php on line
10
bool(true)

Warning: A non-numeric value encountered in /home/guest/sandbox/Solution.php on line
11
...
```

4. Conditional or Ternary Operators

These operators are used to compare two values and take either of the results simultaneously, depending on whether the outcome is TRUE or FALSE. These are also used as a shorthand notation for the *if...else* statement that we will read in the article on decision making.

Syntax:

```
$var = (condition)? value1 : value2;
```

Operator	Name	Operation
?:	Ternary	If the condition is true? then \$x : or else \$y. This means that if the condition is true, then the left result of the colon is accepted otherwise, the result is on the right.

```
<?php
    $x = -12;
    echo ($x > 0) ? 'The number is positive' : 'The number is negative';
?>
```

Output

```
The number is negative
```

5. Assignment Operators

Assignment operators are used to assign values to variables. These operators allow you to assign a value and perform operations in a single step.

Operator	Name	Syntax	Operation
=	Assign	$\$x = \y	Operand on the left obtains the value of the operand on the right
+=	Add then Assign	$\$x += \y	Simple Addition same as $\$x = \$x + \$y$
-=	Subtract then Assign	$\$x -= \y	Simple subtraction same as $\$x = \$x - \$y$
*=	Multiply then Assign	$\$x *= \y	Simple product same as $\$x = \$x * \$y$
/=	Divide, then assign (quotient)	$\$x /= \y	Simple division same as $\$x = \$x / \$y$
%=	Divide, then assign (remainder)	$\$x \% = \y	Simple division same as $\$x = \$x \% \$y$

Example:

```
<?php
// Simple assign operator
$y = 75;
echo $y, "\n";

// Add then assign operator
$y = 100;
$y += 200;
echo $y, "\n";

// Subtract then assign operator
$y = 70;
$y -= 10;
echo $y, "\n";

// Multiply then assign operator
$y = 30;
$y *= 20;
echo $y, "\n";
```

```
// Divide then assign(quotient) operator
$y = 100;
$y /= 5;
echo $y, "\n";

// Divide then assign(remainder) operator
$y = 50;
$y %= 5;
echo $y;

?>
```

Output

```
75
300
60
600
20
0
```

6. Array Operators

These operators are used in the case of arrays. Here are the array operators, along with their syntax and operations, that PHP provides for the array operation.

Operator	Name	Syntax	Operation
+	Union	$\$x + \y	Union of both, i.e., $\$x$ and $\$y$
==	Equality	$\$x == \y	Returns true if both have the same key-value pair
!=	Inequality	$\$x != \y	Returns True if both are unequal
===	Identity	$\$x === \y	Returns True if both have the same key-value pair in the same order and of the same type
!==	Non-Identity	$\$x !== \y	Returns True if both are not identical to each other
<>	Inequality	$\$x <> \y	Returns True if both are unequal

Example:

```
<?php
$x = array("k" => "Car", "l" => "Bike");
$y = array("a" => "Train", "b" => "Plane");

var_dump($x + $y);
var_dump($x == $y) . "\n";
var_dump($x != $y) . "\n";
var_dump($x <> $y) . "\n";
var_dump($x === $y) . "\n";
var_dump($x !== $y) . "\n";

?>
```

Output:

```
array(4) {
  ["k"]=>
  string(3) "Car"
  ["l"]=>
  string(4) "Bike"
  ["a"]=>
  string(5) "Train"
  ["b"]=>
  string(5) "Plane"
}
bool(false)
bool(true)
bool(true)
bool(false)
bool(true)
```

7. Increment/Decrement Operators

These are called the unary operators as they work on single operands. These are used to increment or decrement values.

Operator	Name	Syntax	Operation
++	Pre-Increment	++\$x	First, increment \$x by one, then return \$x
--	Pre-Decrement	--\$x	First, decrement \$x by one, then return \$x
++	Post-Increment	\$x++	First returns \$x, then increment it by one
--	Post-Decrement	\$x--	First, return \$x, then decrement it by one

Example:

```
<?php
$x = 2;
echo ++$x, " First increments then prints \n";
echo $x, "\n";

$x = 2;
echo $x++, " First prints then increments \n";
echo $x, "\n";

$x = 2;
echo --$x, " First decrements then prints \n";
echo $x, "\n";

$x = 2;
echo $x--, " First prints then decrements \n";
echo $x;
?>
```


Output

```

3 First increments then prints
3
2 First prints then increments
3
1 First decrements then prints
1
2 First prints then decrements
1

```

8. String Operators

This operator is used for the concatenation of 2 or more strings using the concatenation operator ('.'). We can also use the concatenating assignment operator ('.=') to append the argument on the right side to the argument on the left side.

Operator	Name	Syntax	Operation
.	Concatenation	<code>\$x.\$y</code>	Concatenated \$x and \$y
<code>.=</code>	Concatenation and assignment	<code>\$x.= \$y</code>	First, it concatenates then assigns, the same as <code>\$x = \$x.\$y</code>

```

<?php
    $x = "Great";
    $y = "Heights";
    $z = "Academy!!!";
    echo $x . $y . $z, "\n";
    $x .= $y . $z;
    echo $x;
?>

```

Output:

```

GreatHeightsAcademy!!!
GreatHeightsAcademy!!!

```

CONTROL STRUCTURES

Control structures determine which code runs and how many times it runs.

They include:

Decision-> if, if-else, if-elseif-else, switch

Repetition-> for, while, do-while, foreach

1. if Statement

The if statement is the simplest form of decision making. It executes a block of code if the specified condition evaluates to true. If the condition evaluates to false, the block of code is skipped.

Syntax:

```
if (condition){
    // if TRUE then execute this code
}
```

Example:

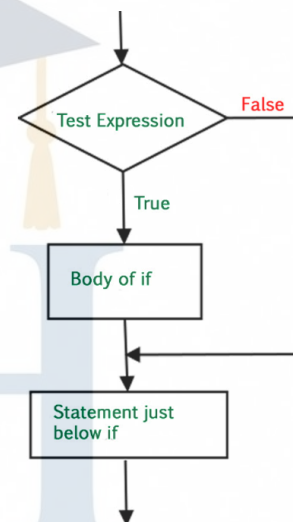
```
<?php
$x = 12;

if ($x > 0) {
    echo "The number is positive";
}
?>
```

Output:

The number is positive

Flowchart:



2. if...else Statement

The if-else statement is an extension of the if statement. It allows you to specify an alternative block of code that is executed when the condition is false. This is useful when there are two mutually exclusive conditions.

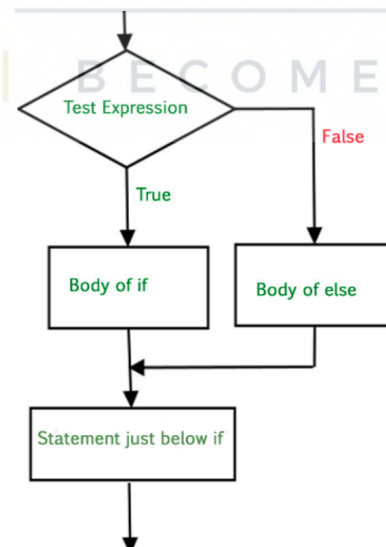
Syntax:

```
if (condition) {
    // if TRUE then execute this code
}
else{
    // if FALSE then execute this code
}
```

Example:

```
<?php
$x = -12;
if ($x > 0) {
    echo "The number is positive";
}
else{
    echo "The number is negative";
}
?>
```

Flowchart:



3. if...elseif...else Statement

In scenarios where you need to evaluate multiple conditions, you can use the if-elseif-else ladder. This construct allows you to check multiple conditions in sequence. The first condition that evaluates to true will execute its corresponding block of code, and all other conditions will be skipped.

Syntax:

```
if (condition) {
    // if TRUE then execute this code
}
elseif {
    // if TRUE then execute this code
}
elseif {
    // if TRUE then execute this code
}
else {
    // if FALSE then execute this code
}
```

Example:

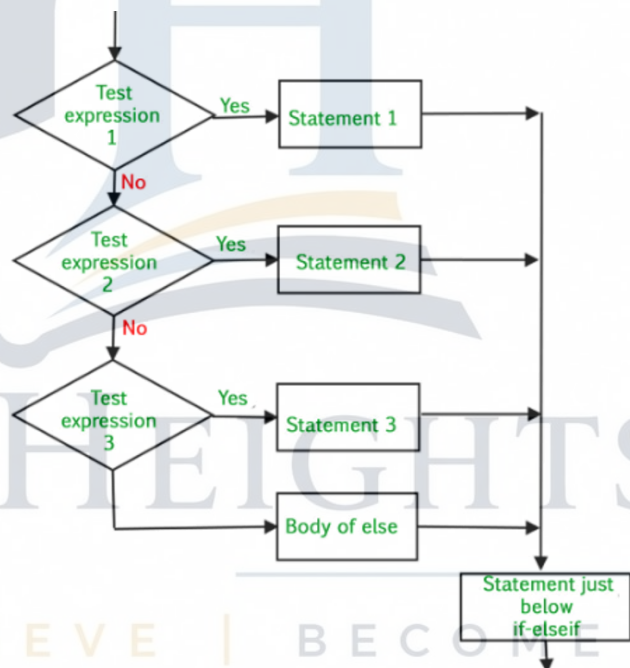
```
<?php
$x = "August";

if ($x == "January") {
    echo "Happy Republic Day";
}
elseif ($x == "August") {
    echo "Happy Independence Day!!!";
}
else{
    echo "Nothing to show";
}
?>
```

Output:

Happy Independence Day!!!

Flowchart:



4. Switch Statement

The "switch" performs in various cases i.e., it has various cases to which it matches the condition and appropriately executes a particular case block. It first evaluates an expression and then compares with the values of each case. If a case matches then the same case is executed. To use switch, we need to get familiar with two different keywords namely, break and default.

- The break statement is used to stop the automatic control flow into the next cases and exit from the switch case.
- The default statement contains the code that would execute if none of the cases match.

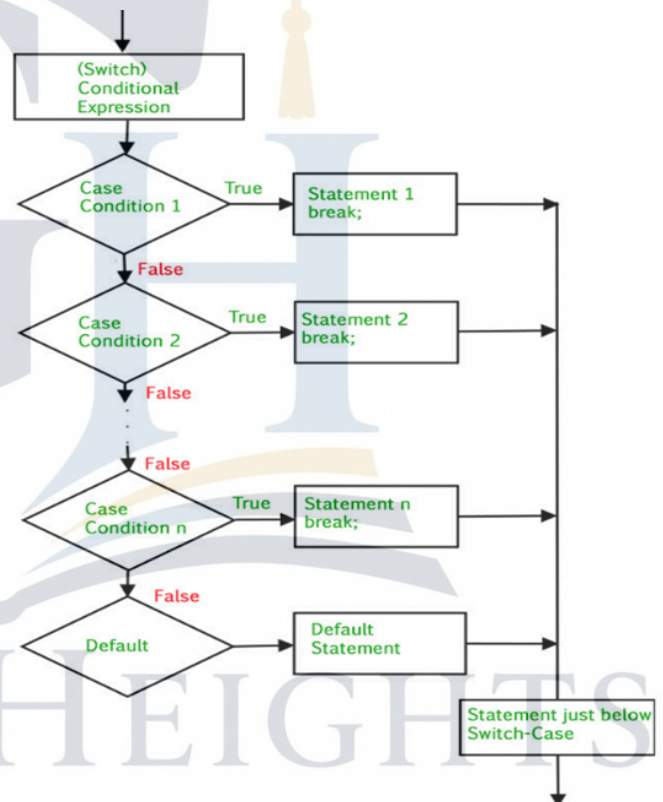
Syntax

```
switch (variable) {
    case value1:
        // Code to be executed if the variable equals value1
        break;
    case value2:
        // Code to be executed if the variable equals value2
        break;
    default:
        // Code to be executed if no cases match
}
```

Example:

Flowchart:

```
<?php
$day = "Monday";
switch ($day) {
    case "Monday":
        echo "Start of the week!";
        break;
    case "Friday":
        echo "End of the week!";
        break;
    case "Saturday":
    case "Sunday":
        echo "Weekend!";
        break;
    default:
        echo "Mid-week!";
}
?>
```



Output

Start of the week!

Note:

- The switch statement compares the variable's value strictly (both type and value).
- If no case matches, the default block is executed (if provided).
- The break statement is crucial; without it, the program will continue to execute subsequent case blocks (a behavior known as "fall-through").

5. Ternary Operators

PHP also provides a shorthand way to perform conditional checks using the ternary operator (?:). This operator allows you to write simple if-else conditions in a compact form.

Syntax

```
(condition) ? if TRUE execute this : otherwise execute this;
```

Example:

```
<?php
$age = 20;
echo ($age >= 18) ? "You are an adult." : "You are a minor.";
?>
```

Output

You are an adult.

PHP Loops

In PHP, Loops are used to repeat a block of code multiple times based on a given condition. PHP provides several types of loops to handle different scenarios, including while loops, for loops, do...while loops, and foreach loops.

Types of Loops in PHP: i) for loop ii) while loop iii) do-while loop iv) foreach loop

1. PHP for loop

PHP for loop is used when you know exactly how many times you want to iterate through a block of code. It consists of three expressions:

- **Initialization:** Sets the initial value of the loop variable.
- **Condition:** Checks if the loop should continue.
- **Increment/Decrement:** Changes the loop variable after each iteration.

Syntax

```
for ( Initialization; Condition; Increment/Decrement ) {
    // Code to be executed
}
```

Example: Printing numbers from 1 to 5 using a for loop.

```
<?php

// Code to illustrate for loop
for ($num = 1; $num <= 5; $num += 1) {
    echo $num . "\n";
}

?>
```

2. PHP while Loop

The while loop is also an entry control loop like for loops. It first checks the condition at the start of the loop, and if it's true then it enters into the loop and executes the block of statements and goes on executing it as long as the condition holds true.

Syntax

```
while ( condition ) {  
    // Code is executed  
}
```

Example: Printing numbers from 1 to 5.

```
<?php  
$num = 1;  
while ($num <= 5) {  
    echo $num . "\n";  
    $num++;  
}  
?>
```

3. PHP do-while Loop

The do-while loop is an exit control loop, which means, it first enters the loop, executes the statements, and then checks the condition. Therefore, a statement is executed at least once using the do...while loop. After executing once, the program is executed as long as the condition holds true.

Syntax

```
do {  
    // Code is executed  
} while ( condition );
```

Example: Printing numbers from 1 to 5.

```
<?php  
$num = 1;  
do {  
    echo $num . "\n";  
    $num++;  
} while ($num <= 5);  
?>
```


4. PHP foreach Loop

This foreach loop is used to iterate over arrays. For every counter of the loop, an array element is assigned, and the next counter is shifted to the next element. It simplifies working with arrays and objects by automatically iterating through each element.

Syntax:

```
foreach ( $array as $value ) {  
    // Code to be executed  
}  
or  
foreach ($array as $key => $value) {  
    // Code to be executed  
}
```

- **\$array:** The array to iterate over.
- **\$value:** The current value of the array element during each iteration.

Example: Iterating through an array

```
<?php  
// foreach loop over an array  
$arr = array (10, 20, 30, 40, 50, 60);  
foreach ($arr as $val) {  
    echo $val . " ";  
}  
echo "\n";  
// foreach loop over an array with keys  
$ages = array(  
    "Anjali" => 25,  
    "Kriti" => 30,  
    "Ayushi" => 22  
);  
foreach ($ages as $name => $age) {  
    echo $name . " => " . $age . "\n";  
}  
?>
```

Output

```
10 20 30 40 50 60  
Anjali => 25  
Kriti => 30  
Ayushi => 22
```


PHP Functions

A function in PHP is a self-contained block of code that performs a specific task. It can accept inputs (parameters), execute a set of statements, and optionally return a value.

- PHP functions allow code reusability by encapsulating a block of code to perform specific tasks.
- Functions can accept parameters and return values, enabling dynamic behavior based on inputs.
- PHP supports both built-in functions and user-defined functions, enhancing flexibility and modularity in code.

```
<?php

// Creating a function that adds two numbers
function addNumbers($a, $b) {

    // code to be executed
    $result = $a + $b;

    // returning the sum
    return $result;
}

// Calling the function
echo addNumbers(5, 10);

?>
```

Syntax

```
function functionName($param1, $param2) {
    // Code to be executed
    return $result; // optional
}
```

Calling a Function in PHP

Once a function is declared, it can be invoked (called) by simply using the function name followed by parentheses.

```
<?php
function sum($a, $b) {
    return $a + $b;
}
echo sum(5, 3); // Outputs: 8
?>
```

Output

8

- The function `sum()` is defined to take two parameters, `$a` and `$b`, and return their sum.
- The function is called with 5 and 3 as arguments, and it returns the result, which is 8.

Types of Functions in PHP

PHP has two types of functions:

1. User-Defined Functions

A user-defined function is created to perform a specific task as per the developer's need. These functions can accept parameters, perform computations, and return results.

```
<?php
function addNumbers($a, $b) {
    return $a + $b;
}
echo addNumbers(5, 3); // Output: 8
?>
```

Output

8

- The function `addNumbers` is defined by the user to take two parameters, `$a` and `$b`, and returns their sum (`$a + $b`).
- The function is called with 5 and 3 as arguments, so it adds these numbers together.

2. Built-in Functions

PHP comes with many built-in functions that can be directly used in your code. For example, `strlen()`, `substr()`, `array_merge()`, `date()` and etc are built-in PHP functions. These functions provide useful functionalities, such as string manipulation, date handling, and array operations, without the need to write complex logic from scratch.

```
<?php
    $str = "Hello, World!";
    echo strlen($str); // Output: 13
?>
```

Output

13

- A string variable \$str is defined with the value "Hello, World!".
- The strlen() function is the built-in function in PHP that is used to calculate the length of the string \$str.

PHP Function Parameters and Arguments

Functions can accept input values, known as parameters or arguments.

- Parameters are variables defined in the function declaration that accept values.
- Arguments are the actual values passed to the function when it is called.

These values can be passed to the function in two ways:

1. Passing by Value

When you pass a value by value, the function works with a copy of the argument, so the original value remains unchanged.

```
<?php
function add($x, $y) {
    $x = $x + $y;
    return $x;
}

echo add(2, 3); // Outputs: 5
?>
```

Output

5

- The function add() is defined with parameters \$x and \$y.
- The function adds \$x and \$y and returns the sum.
- The function is called with values 2 and 3, and the result 5 is printed.

2. Passing by Reference

By passing an argument by reference, any changes made inside the function will affect the original variable outside the function.

```
<?php
function addByRef(&$x, $y) {
    $x = $x + $y;
}
$a = 5;
addByRef($a, 3);
echo $a; // Outputs: 8
?>
```

Output

8

- The function addByRef() is defined with \$x passed by reference.
- The function modifies \$x directly, adding \$y to it.
- After calling the function with \$a = 5 and 3, the value of \$a becomes 8.

Returning Values in PHP Functions

Functions in PHP can return a value using the return statement. The return value can be any data type such as a string, integer, array, or object. If no return statement is provided, the function returns null by default.

- The function multiply() is defined to return the product of \$a and \$b.
- The function is called with 4 and 5 as arguments.
- The result 20 is returned and printed using echo.

```
<?php
function multiply($a, $b) {
    return $a * $b;
}
$result = multiply(4, 5); // $result will be 20
echo $result; // Output: 20
?>
```

Output

20

Anonymous Functions (Closures)

PHP supports anonymous functions, also known as closures. These functions do not have a name and are often used for passing functions as arguments to other functions.

```
<?php
    $greet = function($name) {
        echo "Hello, " . $name . "!";
    };
    $greet("GFG");
?>
```

Output

Hello, GFG!

- An anonymous function is defined and assigned to the variable \$greet.
- The function takes \$name as a parameter and prints a greeting message.
- The function is called with the argument "GFG", and the greeting "Hello, GFG" is printed.

Recursion in PHP

Recursion is a process in which a function calls itself. It is often used in situations where a task can be broken down into smaller, similar tasks.

```
<?php
function factorial($n) {
    if ($n == 0) {
        return 1;
    } else {
        return $n * factorial($n - 1);
    }
}
echo factorial(5);
?>
```

Output

120

- The function factorial() is defined to calculate the factorial of a number \$n using recursion.
- If \$n is 0, the function returns 1 (base case); otherwise, it calls itself with \$n - 1.
- When factorial(5) is called, the function recursively multiplies 5 * 4 * 3 * 2 * 1, returning 120.

Common PHP Functions

Here are some common PHP functions you should know:

1. String Functions

- `strlen()`: Returns the length of a string.
- `strtoupper()`: Converts a string to uppercase.
- `strtolower()`: Converts a string to lowercase.
- `substr()`: Returns a part of a string.

2. Array Functions

- `array_push()`: Adds elements to the end of an array.
- `array_pop()`: Removes the last element of an array.
- `array_merge()`: Merges two or more arrays.

3. Date and Time Functions

- `date()`: Returns the current date or time.
- `strtotime()`: Converts a string into a Unix timestamp.
- `time()`: Returns the current Unix timestamp.

Including Files

When applications become large, we don't want everything inside one PHP file. We can divide code into multiple files.

For example:

project

├─ index.php

├─ header.php

├─ footer.php

└─ database.php

PHP provides:

`include`

`require`

`include_once`

`require_once`

include

`include "header.php";`

PHP includes the contents of that file.

If the file cannot be found, `include` produces a warning, and execution can generally continue.

require

```
require "database.php";
```

If the required file cannot be loaded, PHP generates a fatal error and execution stops.

Difference

include	require
Missing file → warning	Missing file → fatal error
Execution generally continues	Execution stops
Useful for optional components	Useful for essential components

include_once and require_once

These prevent the same file from being included multiple times.

```
include_once "header.php";
```

or:

```
require_once "database.php";
```

require_once is commonly useful for database/configuration files.

PHP Arrays

Arrays are one of the most important data structures in PHP. They allow you to store multiple values in a single variable. PHP arrays can hold values of different types, such as strings, numbers, or even other arrays. Understanding how to use arrays in PHP is important for working with data efficiently.

- PHP offers many built-in array functions for sorting, merging, searching, and more.
- PHP Arrays can store values of different types (e.g., strings, integers, objects, or even other arrays) in the same array.
- They are dynamically sized.
- They allow you to store multiple values in a single variable, making it easier to manage related data.

Types of Arrays in PHP

There are three main types of arrays in PHP:

1. Indexed Arrays

Indexed Arrays use numeric indexes starting from 0. These arrays are ideal when you need to store a list of items where the order matters.

```
<?php
    $fruits = array("apple", "banana", "cherry");
    echo $fruits[0]; // Outputs: apple
?>
```

Output

apple

2. Associative Arrays

Associative Arrays use named keys, which are useful when you want to store data with meaningful identifiers instead of numeric indexes.

```
<?php
    $person = array("name" => "GFG", "age" => 30, "city" => "New York");
    echo $person["name"];
?>
```

Output

GFG

3. Multidimensional Arrays

Multidimensional Arrays are arrays that contain other arrays as elements. These are used to represent more complex data structures, such as matrices or tables.

```
<?php
    $students = array(
        "Anjali" => array("age" => 25, "grade" => "A"),
        "GFG" => array("age" => 22, "grade" => "B")
    );
    echo $students["GFG"]["age"];
?>
```

Output

22

Creating Array in PHP

In PHP, arrays can be created using two main methods:

1. Using the array() function

The traditional way of creating an array is using the array() function.

```
$fruits = array("apple", "banana", "cherry");
```

2. Using short array syntax ([])

In PHP 5.4 and later, you can use the shorthand [] syntax to create arrays.

```
$fruits = ["apple", "banana", "cherry"];
```

You can also create associative arrays by specifying custom keys:

```
$person = ["name" => "GFG", "age" => 30];
```

Accessing and Modifying Array Elements

1. Accessing Array Elements

You can access individual elements in an array using their index (for indexed arrays) or key (for associative arrays).

- Accessing Indexed Array:

```
$fruits = ["apple", "banana", "cherry"];
```

```
echo $fruits[0]; // Outputs: apple
```

- Accessing Associative Array:

```
$person = ["name" => "GFG", "age" => 30];
```

```
echo $person["name"]; // Outputs: GFG
```

2. Modifying Array Elements

You can modify an existing element by assigning a new value to a specific index or key.

- Modifying Indexed Array Element:

```
$fruits = ["Apple", "Banana", "Cherry"];
```

```
$fruits[1] = "Mango"; // Changes "Banana" to "Mango"
```

```
echo $fruits[1]; // Outputs: Mango
```

- Modifying Associative Array Element:

```
$person = ["name" => "GFG", "age" => 25];
```

```
$person["age"] = 26; // Updates the age to 26
```

```
echo $person["age"]; // Outputs: 26
```

Adding and Removing Array Items

1. Adding Array Elements

You can add new elements to an array using the following methods:

- `array_push()`: Adds elements to the end of an indexed array.

```
$fruits = ["apple", "banana"];
```

```
array_push($fruits, "cherry"); // Adds "cherry" to the end
```



- `array_unshift()`: Adds elements to the beginning of an indexed array.

```
array_unshift($fruits, "pear"); // Adds "pear" to the beginning
```

- Direct assignment: Adds an element to an associative array.

```
$person["city"] = "New York";
```

2. Removing Array Elements

To remove items from an array, you can use several functions:

- `array_pop()`: Removes the last element from an indexed array.

```
array_pop($fruits);
```

- `array_shift()`: Removes the first element from an indexed array.

```
array_shift($fruits);
```

- `unset()`: Removes a specific element from an array by key or index.

```
unset($fruits[2]); // Removes the element with index 2
```

Array Functions

PHP provides a wide range of built-in functions to work with arrays. Here are some common array functions:

- Array Merge: The `array_merge()` function combines two or more arrays into one.

```
$array1 = [1, 2, 3];
```

```
$array2 = [4, 5, 6];
```

```
$merged = array_merge($array1, $array2);
```

```
print_r($merged); // Outputs: [1, 2, 3, 4, 5, 6]
```

- Array Search: The `in_array()` function checks if a specific value exists in an array.

```
$fruits = ["Apple", "Banana", "Cherry"];
```

```
if (in_array("Banana", $fruits)) {
```

```
    echo "Banana is in the array!";
```

```
}
```

- Array Sort: The `sort()` function sorts an indexed array in ascending order.

```
$numbers = [3, 1, 4, 1, 5];
```

```
sort($numbers);
```

```
print_r($numbers); // Outputs: [1, 1, 3, 4, 5]
```

Array Iteration

You can loop through arrays using loops such as foreach or for.

- Using foreach Loop:

```
$fruits = ["Apple", "Banana", "Cherry"];  
foreach ($fruits as $fruit) {  
    echo $fruit . "<br>";  
}
```

- Using for Loop:

```
$numbers = [1, 2, 3, 4];  
for ($i = 0; $i < count($numbers); $i++) {  
    echo $numbers[$i] . "<br>";  
}
```

PHP - Array Sorting Functions

The items in an array can be sorted in alphabetical or numerical order, descending or ascending.

Here are the main PHP array sorting functions:

- `sort()` - sorts an indexed array in ascending order
- `rsort()` - sorts an indexed array in descending order
- `asort()` - sorts an associative array in ascending order, according to the value
- `ksort()` - sorts an associative array in ascending order, according to the key
- `arsort()` - sorts an associative array in descending order, according to the value
- `krsort()` - sorts an associative array in descending order, according to the key

1. `sort()` — Ascending Order

Sorts an indexed array in ascending order.

Example

```
<?php  
$numbers = [50, 20, 40, 10, 30];  
sort($numbers);  
print_r($numbers);  
?>
```



Output

Array

```
(  
    [0] => 10  
    [1] => 20  
    [2] => 30  
    [3] => 40  
    [4] => 50  
)
```

2. **rsort()** — Descending Order

Sorts an indexed array in descending order.

```
<?php  
$numbers = [50, 20, 40, 10, 30];  
rsort($numbers);  
print_r($numbers);  
?>
```

Output

Array

```
(  
    [0] => 50  
    [1] => 40  
    [2] => 30  
    [3] => 20  
    [4] => 10  
)
```

Associative Array Sorting

Consider this associative array:

```
$students = [
```




```
"Rahul" => 85,  
"Priya" => 92,  
"Amit" => 78,  
"Sneha" => 88  
];
```

Here:

Key	Value
-----	-------

Rahul	→ 85
-------	------

Priya	→ 92
-------	------

Amit	→ 78
------	------

Sneha	→ 88
-------	------

Now we can sort either by value or by key.

3. asort() — Value Ascending

Sorts an associative array according to its values in ascending order.

```
<?php
```

```
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];
```

```
];
```

```
asort($students);
```

```
print_r($students);
```

```
?>
```

Output

Array

(

[Amit] => 78



```
[Rahul] => 85  
[Sneha] => 88  
[Priya] => 92  
)
```

Notice that the keys remain attached to their values.

```
Amit → 78  
Rahul → 85  
Sneha → 88  
Priya → 92
```

4. **arsort()** — Value Descending

Sorts an associative array according to its values in descending order.

```
<?php  
$students = [  
    "Rahul" => 85,  
    "Priya" => 92,  
    "Amit" => 78,  
    "Sneha" => 88  
];  
arsort($students);  
print_r($students);  
?>
```

Output

Array

```
(  
    [Priya] => 92  
    [Sneha] => 88  
    [Rahul] => 85  
    [Amit] => 78  
)
```

Again, the keys are preserved.

BUILD | BELIEVE | BECOME



5. ksort() — Key Ascending

Sorts an associative array according to its keys in ascending order.

```
<?php
$students = [
    "Rahul" => 85,
    "Priya" => 92,
    "Amit" => 78,
    "Sneha" => 88
];
ksort($students);
print_r($students);
?>
```

Output

```
Array
(
    [Amit] => 78
    [Priya] => 92
    [Rahul] => 85
    [Sneha] => 88
)
```

Notice that we are sorting the names, not the marks.

6. krsort() — Key Descending

Sorts an associative array according to its keys in descending order.

```
<?php
$students = [
    "Rahul" => 85,
    "Priya" => 92,
    "Amit" => 78,
    "Sneha" => 88
];
```



```
krsort($students);  
print_r($students);  
?>
```

Output

Array

```
(  
    [Sneha] => 88  
    [Rahul] => 85  
    [Priya] => 92  
    [Amit] => 78  
)
```

String Manipulation

String: A string is a sequence of characters.

```
$name = "Great Heights";
```

Strings are extremely important in PHP because web applications process a lot of textual data:

- Names
- Emails
- Addresses
- Passwords
- Messages
- URLs
- Search terms

String Concatenation

Concatenation means joining strings together.

PHP uses: `.` for concatenation.

Example:

```
$firstName = "Rahul";  
$lastName = "Das";  
echo $firstName . " " . $lastName;
```

Output:

Rahul Das

String Interpolation

Double-quoted strings can directly contain variables.

```
$name = "Rahul";  
echo "Hello $name";
```

Output:

Hello Rahul

With curly braces, you can make variable boundaries clearer:

```
echo "Hello {$name}";
```

String Functions

Function	Purpose	Example	Result
strlen()	Returns the length of a string	strlen("Hello")	5
str_word_count()	Counts the number of words	str_word_count("Hello World")	2
strtoupper()	Converts string to uppercase	strtoupper("hello")	HELLO
strtolower()	Converts string to lowercase	strtolower("HELLO")	hello
ucfirst()	Converts the first character to uppercase	ucfirst("hello")	Hello
ucwords()	Converts the first character of each word to uppercase	ucwords("hello world")	Hello World
trim()	Removes whitespace from both ends	trim(" Hello ")	Hello
ltrim()	Removes whitespace from the left side	ltrim(" Hello")	Hello
rtrim()	Removes whitespace from the right side	rtrim("Hello ")	Hello
str_replace()	Replaces one string with another	str_replace("PHP","Java","I love PHP")	I love Java

strpos()	Finds the position of the first occurrence of text	strpos("Hello World","World")	6
strrpos()	Finds the position of the last occurrence	strrpos("Hello Hello","Hello")	6
substr()	Extracts a portion of a string	substr("Hello World",0,5)	Hello
str_contains()	Checks whether a string contains specific text	str_contains("Hello World","World")	true
explode()	Splits a string into an array	explode(",","PHP,MySQL,HTML")	["PHP","MySQL","HTML"]
implode()	Joins array elements into a string	implode(",","["PHP","MySQL"]")	PHP, MySQL
str_split()	Splits a string into individual characters	str_split("Hello")	["H","e","l","l","o"]
strcmp()	Compares two strings	strcmp("Hello","Hello")	0
strcasecmp()	Compares two strings without considering case	strcasecmp("Hello","hello")	0
str_repeat()	Repeats a string a specified number of times	str_repeat("Hi ",3)	Hi Hi Hi
str_pad()	Adds characters to make a string a specified length	str_pad("PHP",6,"*")	PHP***
htmlspecialchars()	Converts special HTML characters into safe HTML entities	htmlspecialchars(" Hi ")	<h1>Hi</h1>